

Lab 3 — Architectural Justification

Restaurant Table Booking & Pre-Ordering App | Problem Statement #36

Name: Varun M L

SRN: PES1UG25AM816

Section: G

Architecture Selection

We chose **Microservices Architecture** for the Restaurant Table Booking & Pre-Ordering System

Architectural Style Analysis

Layered Architecture provides clear separation of concerns but makes independent scaling difficult. Client-Server offers simpler centralized management but can become a scalability bottleneck and a single point of failure. Microservices supports independent scaling, fault isolation, and deployment, making it the most suitable choice for this restaurant system's real-time table synchronization and compute-heavy kitchen processing.

Two Scenario-Related Reasons

1. Independent scaling & fault isolation — The Kitchen Prep Timeline calculation is compute-heavy during peak dinner hours, while the Floor-Plan Dashboard must sync table status across all manager terminals in under 500 ms (NFR-001). Splitting these into separate services lets each scale independently under load, and a slowdown or failure in the Menu Catalog service does not block active reservations or table-status updates.

2. Independent deployment aligned to backlog epics — Each of our four sprint epics — Table Reservation & Floor Plan, Menu Browsing & Pre-Ordering, Manager Floor-Plan Dashboard, and Security & Data Protection — maps to a bounded, independently deployable service. A change to the kitchen-prep algorithm or menu pricing can ship without redeploying the Reservation Manager or Payment Service, lowering release risk during live service hours.

Security Advantage

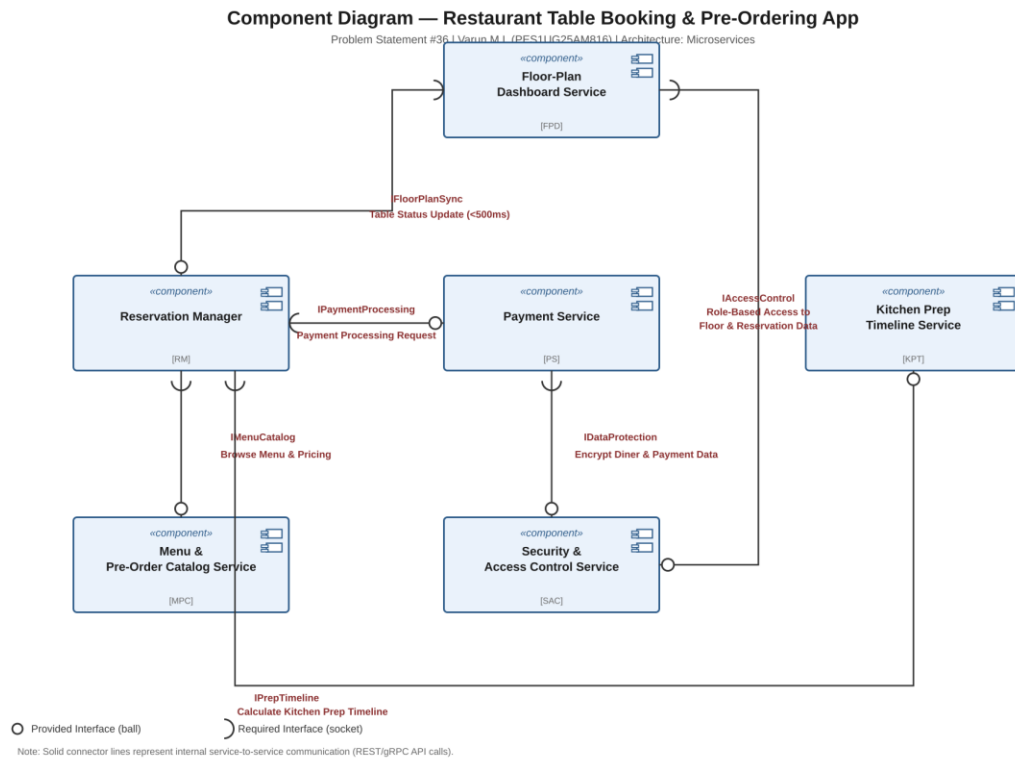
Diner personal and payment data (NFR-002) is isolated behind a dedicated Security & Access Control Service that owns encryption and role-based access enforcement. Other services never touch raw payment data directly — they call it only through the IDataProtection and IAccessControl interfaces — which shrinks the attack surface and limits the blast radius if any single service is compromised.

Performance Benefit

Because real-time table-status sync is handled entirely by its own Floor-Plan Dashboard Service, it never competes for resources with heavier, latency-tolerant work such as kitchen-prep-timeline computation. This lets us scale Floor-Plan Dashboard replicas independently to reliably meet the sub-500ms sync requirement across all manager terminals, even during peak load.

Interface Technologies: Payment Processing — REST API; Menu Catalog — REST API; Floor-Plan Sync — WebSocket; Data Protection — Internal API; Access Control — Internal API; Kitchen Prep Timeline — REST API.

Component Diagram



6 components — Reservation Manager, Payment Service, Menu & Pre-Order Catalog Service, Kitchen Prep Timeline Service, Floor-Plan Dashboard Service, Security & Access Control Service — connected by 6 labelled interfaces using provided/required ball-and-socket notation. Interfaces are labelled with communication technology such as REST API, WebSocket, and internal API.